

A3Q3

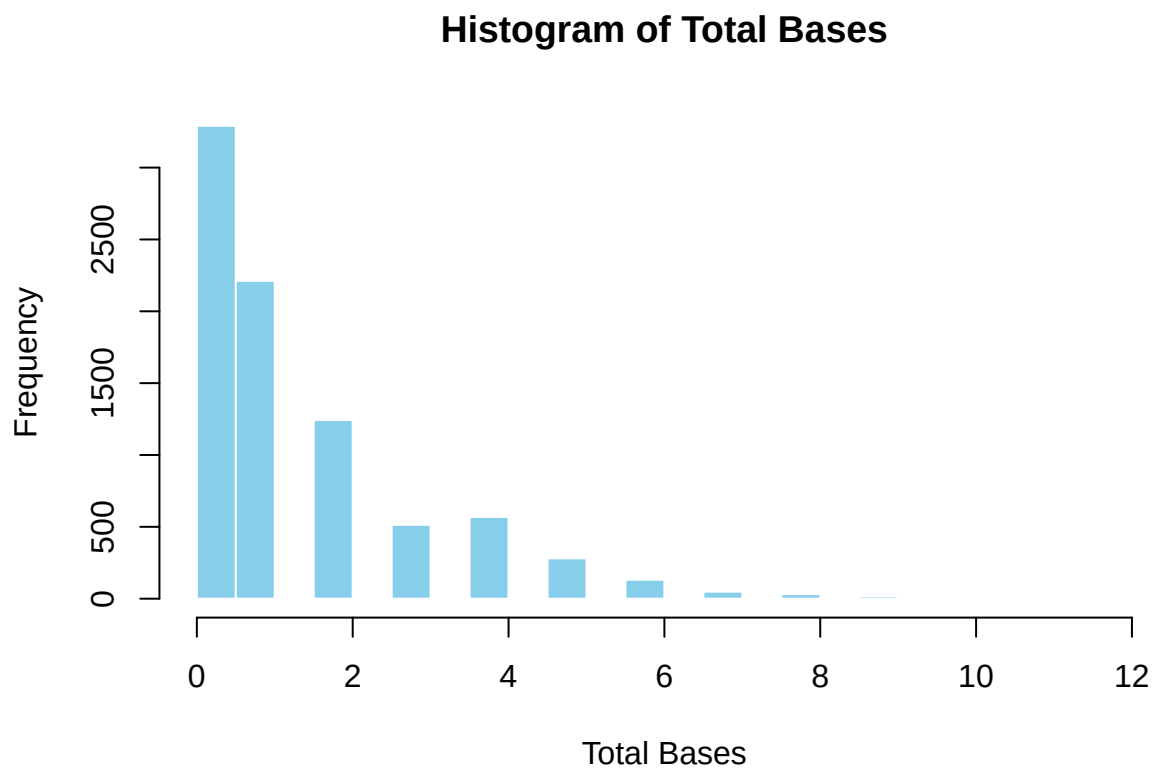
Lily Li

2025-07-05

Q3

(a)

```
mlb_bs <- read.csv("MLB_Batting_Boxscores_2024-04.csv")  
y <- mlb_bs$total_bases  
hist(y, breaks = 30, main = "Histogram of Total Bases", xlab = "Total Bases",  
     col = "skyblue", border = "white")
```



The zero-inflated Poisson distribution is a suitable model for this data, because the data shows a large spike at 0, and with a high variability.

(b)

```
E <- mean(y)
V <- var(y)

pi_hat_MM <- (V - E) / (E^2 + V - E)
lambda_hat_MM <- E + V/E - 1

pi_hat_MM
```

```
## [1] 0.4301142
```

```
lambda_hat_MM
```

```
## [1] 2.433774
```

(e)

```
rho <- function(theta, y) {
  pi <- theta[1]
  lambda <- theta[2]
  Z <- sum(y == 0)
  N <- length(y)
  sum_y <- sum(y[y > 0])
  logfact_y <- sum(lfactorial(y[y > 0]))

  log_likelihood <- Z * log(pi + (1 - pi) * exp(-lambda)) + (N-Z) * log(1-pi) +
    sum_y * log(lambda) - (N - Z) * lambda - logfact_y

  return(-log_likelihood)
}

gradient <- function(theta, y) {
  pi <- theta[1]
  lambda <- theta[2]
  Z <- sum(y == 0)
  N <- length(y)
  sum_y <- sum(y[y > 0])

  denom <- pi + (1-pi) * exp(-lambda)
  d_pi <- Z * (1 - exp(-lambda)) / denom - (N-Z)/(1-pi)
  d_lambda <- -Z * (1-pi) * exp(-lambda) / denom + sum_y / lambda - (N-Z)

  return(-c(d_pi, d_lambda))
}
```

(f)

```
testConvergence <- function(thetaNew, thetaOld, tolerance = 1E-10, relative=FALSE) {
  sum(abs(thetaNew - thetaOld)) < if (relative) tolerance * sum(abs(thetaOld)) else tolerance
}

gridLineSearch <- function(theta, rhoFn, d, lambdaStepsize = 0.01, lambdaMax = 1, y) {
  ## grid of lambda values to search
  lambdas <- seq(from = 0, by = lambdaStepsize, to = lambdaMax)
  ## line search
  rhoVals <- sapply(lambdas, function(lambda) {rhoFn(theta - lambda * d, y)})
  ## Return the lambda that gave the minimum
  lambdas[which.min(rhoVals)]
}

gradientDescent <- function(theta = c(0,1), rhoFn, gradientFn, lineSearchFn,
  testConvergenceFn, maxIterations = 100,
  tolerance = 1E-6, relative = FALSE,
  lambdaStepsize = 0.01, lambdaMax = 0.5, y) {

  converged <- FALSE
  i <- 0

  while (!converged & i <= maxIterations) {

    g <- gradientFn(theta, y) ## gradient
    glength <- sqrt(sum(g^2)) ## gradient direction

    if (glength > 0) {
      d <- g / glength
    } else {
      d <- rep(0, length(theta)) # zero direction if flat gradient
    }

    lambda <- lineSearchFn(theta, rhoFn, d, lambdaStepsize = lambdaStepsize,
      lambdaMax = lambdaMax, y = y)

    thetaNew <- theta - lambda * d

    converged <- testConvergenceFn(thetaNew, theta,
      tolerance = tolerance,
      relative = relative)

    theta <- thetaNew
    i <- i + 1
  }
  ## Return last value and whether converged or not
  list(theta = theta, converged = converged, iteration = i, fnValue = rhoFn(theta, y))
}

theta0 <- c(0, 3)
theta1 <- c(0.2, 5)
theta2 <- c(0.4, 7)
```

```

theta3 <- c(0.6, 10)

result0 <- gradientDescent(theta0, rho, gradient, gridLineSearch, testConvergence,
                           maxIterations = 200, y = y)
result1 <- gradientDescent(theta1, rho, gradient, gridLineSearch, testConvergence,
                           maxIterations = 200, y = y)
result2 <- gradientDescent(theta2, rho, gradient, gridLineSearch, testConvergence,
                           maxIterations = 200, y = y)
result3 <- gradientDescent(theta3, rho, gradient, gridLineSearch, testConvergence,
                           maxIterations = 200, y = y)

results <- list("(0,3)" = result0, "(0.2,5)" = result1,
               "(0.4,7)" = result2, "(0.6,10)" = result3)

for(name in names(results)) {
  res <- results[[name]]

  cat("\n-----\n")
  cat("Starting point:", name, "\n")
  cat("Converged:", res$converged, "\n")
  cat("Iterations:", res$iteration, "\n")
  cat("Minimum_rho_func:", round(res$fnValue, 3), "\n")
  cat("Estimated Parameters:\n")
  cat("  pi = ", round(res$theta[1], 3), "\n")
  cat("  lambda = ", round(res$theta[2], 3), "\n")
}

```

```

##
## -----
## Starting point: (0,3)
## Converged: TRUE
## Iterations: 19
## Minimum_rho_func: 13919.54
## Estimated Parameters:
##   pi = 0.297
##   lambda = 1.99
##
## -----
## Starting point: (0.2,5)
## Converged: TRUE
## Iterations: 34
## Minimum_rho_func: 13919.26
## Estimated Parameters:
##   pi = 0.3
##   lambda = 1.974
##
## -----
## Starting point: (0.4,7)
## Converged: TRUE
## Iterations: 62
## Minimum_rho_func: 13919.43
## Estimated Parameters:

```

```
##    pi = 0.291
##    lambda = 1.972
##
## -----
## Starting point: (0.6,10)
## Converged: TRUE
## Iterations: 68
## Minimum_rho_func: 13919.45
## Estimated Parameters:
##    pi = 0.292
##    lambda = 1.976
```